

Лекция 6. Контроль версий конфигураций и сценариев: Git

Цель лекции

Понять Git как журнал технических изменений администратора

Различать Working Tree, Staging Area и Repository

Безопасно фиксировать изменения Bash-сценариев, конфигураций и документации

Исключать секреты и временные файлы из commit

Понимать, почему Git не заменяет backup и config management

Учебные вопросы

1. Git в работе системного администратора
2. Working Tree, Staging Area и Repository
3. status, diff, add, commit, log и show
4. Restore и базовое ветвление
5. .gitignore и структура инфраструктурного репозитория
6. Запрет хранения секретов
7. Remote Repository и отличие Git от Backup

Учебный сценарий лекции

Студент создает репозиторий linux-infrastructure

Добавляет scripts/server-health.sh из прошлой лекции

Проверяет сценарий перед commit

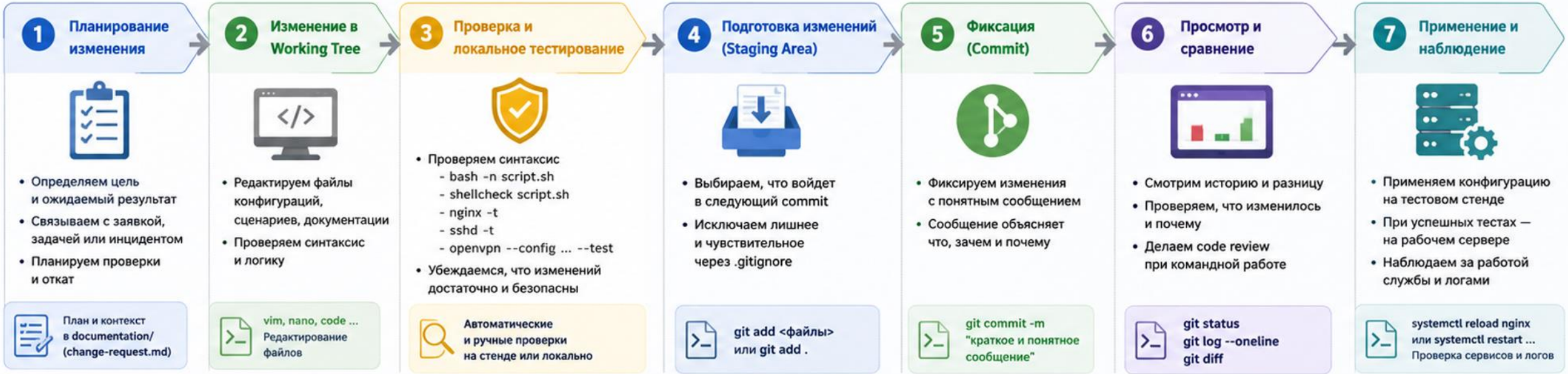
Создает понятный commit с изменением

Исправляет ошибку, когда секрет случайно попал в

Staging Area

Схема: Git в жизненном цикле изменения конфигурации

Git помогает администратору безопасно вносить изменения, проверять их, документировать и при необходимости откатывать.



Результат

- Изменение проверяемое: есть проверки и тесты
- Изменение объяснимое: есть commit и причина
- Изменение обратимое: можно вернуться к рабочему состоянию
- История хранится в безопасном удаленном репозитории

ВАЖНО
Никогда не храните пароли, ключи и live-конфигурации из /etc!
Используйте примеры и шаблоны, .gitignore и безопасные практики.

Типичная структура репозитория linux-infrastructure

linux-infrastructure/

- scripts/
- configs/
- documentation/
- tests/
- .gitignore
- README.md

- # Bash-сценарии и утилиты
- # Конфигурации без секретов
- # Документация (MD)
- # Тесты и проверки

Примеры предварительных проверок

BASH

bash -n script.sh

shellcheck

shellcheck script.sh

nginx

nginx -t

SSH

sshd -t

openvpn

openvpn --config ... --test

Откат (если что-то пошло не так)

Найти стабильный commit в git log

git checkout <commit> или git restore <файл>

Применить конфигурацию и перезапустить службу

Проверить работу службы

Планирование

Редактирование

Проверка

Подготовка

Commit

Проверка изменений

Применение

1. Git для администратора фиксирует технические изменения

Git - система контроля версий для текстовых файлов

Для администратора Git полезен как история изменений конфигураций, сценариев и документации

Git показывает, кто, когда и зачем изменил файл

Git помогает вернуть прежнюю версию текстового файла

Git - это журнал технических изменений, а не только инструмент программиста

Базовая цепочка работы с изменением

Изменение → Проверка → Git Diff → Commit →

История → Возможность восстановления

Изменение сначала проверяют технически

git diff показывает, что именно поменялось

commit фиксирует осмысленный набор изменений

История помогает понять причину и вернуть файл при ошибке

Git не заменяет техническую проверку

Git показывает, что изменилось

Git не доказывает, что служба работает

Перед commit сценария: `bash -n scripts/server-health.sh`

Рекомендуемо для Bash: `shellcheck scripts/server-health.sh`

Перед commit Nginx-конфигурации: `sudo nginx -t`

Перед commit SSH-конфигурации: `sudo sshd -t`

Git, Backup и Config Management решают разные задачи

Git → история текстовых файлов

Backup → восстановление данных и состояния после потери

Config Management → автоматическое применение нужного состояния

Ansible можно рассматривать как пример config management

Git не заменяет backup, потому что не хранит весь сервер и не гарантирует восстановление данных

Что администратору имеет смысл хранить в Git

Bash-сценарии: `scripts/server-health.sh`

Шаблоны конфигураций:

`configs/nginx/app.conf.example`

Документацию: `README.md` и инструкции по применению

Примеры параметров: `.env.example`

Тестовые файлы и лабораторные примеры без секретов

2. Working Tree, Staging Area и Repository нужно различать

Working Tree - файлы, которые студент редактирует сейчас

Staging Area - выбранные изменения для будущего commit

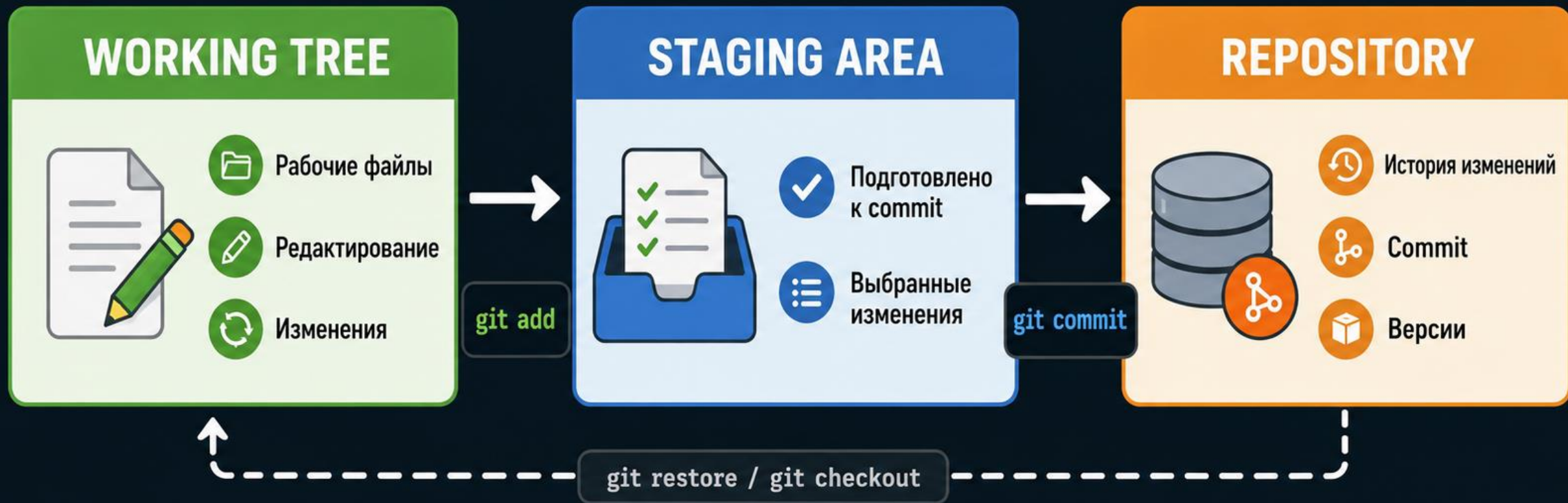
Repository - история commit внутри .git

git add переносит изменение в Staging Area

git commit сохраняет staged-изменения в Repository



WORKING TREE - STAGING AREA - REPOSITORY



Изменил файл →



`git status /
git diff` →



`git add` →



`git commit`

- **Working Tree** = где редактируют
- **Staging Area** = что войдёт в commit
- **Repository** = история проекта

Состояния файла должны быть понятны без лишней терминологии

Untracked - Git еще не отслеживает файл

Modified - отслеживаемый файл изменен в Working Tree

Staged - изменение выбрано для будущего commit

Committed / unchanged - изменение уже сохранено в истории

Для junior этих четырех состояний достаточно

Создание инфраструктурного репозитория

```
mkdir linux-infrastructure
```

```
cd linux-infrastructure
```

```
git init -b main
```

Если `git init -b main` не поддерживается: `git init`

Затем выполнить: `git branch -M main`

Репозиторий создают отдельно, а не прямо в `/etc`

Структура репозитория должна помогать эксплуатации

scripts/ - административные сценарии

configs/ - шаблоны и примеры конфигураций

documentation/ - инструкции и отчеты

examples/ - учебные примеры

tests/ - проверки и тестовые данные

README.md и .gitignore - обязательные служебные файлы

Репозиторий хранит безопасные текстовые артефакты

Git не должен хранить live-сервер целиком

Git не должен хранить дампы, PCAP и приватные ключи

Git хорошо подходит для текстовых сценариев и шаблонов

Перед переносом файла из /etc нужно убрать секреты и привязку к конкретному серверу

README должен объяснять, как применить файл безопасно

3. `git status` - первая и последняя команда рабочего цикла

`git status` показывает состояние Working Tree и Staging Area

`git status --short` показывает компактный вариант

Привычка: начал работу → `git status`

Привычка: закончил работу → `git status`

Перед `commit` нужно видеть, какие файлы попадут в историю

git diff показывает смысл изменения

git diff показывает изменения в Working Tree

git diff --staged показывает то, что войдет в commit

git show HEAD показывает последний commit

Diff нужно читать до git add и после git add

Сложные варианты HEAD~1 и диапазоны commit в этой лекции не являются обязательными

git add должен быть точным

Безопасный пример: `git add scripts/server-health.sh`

Перед add выполнить `git status` и `git diff`

`git add .` может случайно добавить лишнее

Особенно опасны секреты, дампы, PCAP и временные файлы

Администратор добавляет в `commit` только осознанные изменения

Дополнительно: `git add -p` добавляет часть изменений

`git add -p` позволяет добавить только часть изменений файла

Это полезно, если в одном файле сделаны разные независимые правки

Для первой лаборатории достаточно обычного `git add file`

Главная идея: `commit` должен быть осмысленным и не смешивать все подряд

`git add -p` не является обязательной командой junior-уровня

Commit фиксирует проверенное изменение

`git commit -m "Add server health check"`

Хорошее сообщение: Add route for OpenVPN client network

Плохое сообщение: update

Плохое сообщение: fix

Commit message должен объяснять изменение

Перед commit нужна техническая проверка файла

git log и git show помогают читать историю

git log --oneline показывает краткую историю commit

git log --oneline --graph --decorate показывает ветки и указатели

git show HEAD показывает последний commit целиком

История нужна для ответа: что изменилось и почему

Не нужно перегружать лекцию большим набором log-опций

Первый рабочий цикл администратора

```
git status
```

```
git diff
```

```
bash -n scripts/server-health.sh
```

```
git add scripts/server-health.sh
```

```
git diff --staged
```

```
git commit -m "Add server health check"
```

```
git log --oneline
```

```
git show HEAD
```

4. Restore отменяет незакоммиченные изменения

```
git restore scripts/server-health.sh
```

Команда отменяет незакоммиченные изменения файла

Перед restore обязательно выполнить `git status`

Перед restore обязательно выполнить `git diff`

`git restore` может уничтожить полезные правки, если запускать его без проверки

Unstage убирает файл из будущего commit

```
git restore --staged secrets/client.key
```

Команда убирает файл из Staging Area

Файл остается в рабочем каталоге

Это полезно, если секрет случайно попал в staging

После unstage нужно добавить правило в .gitignore

Старую версию файла можно взять из истории

Git позволяет получить старую версию файла из commit

Для junior достаточно понимать саму возможность восстановления

Глубокая тема `git restore --source` переносится в дополнительный материал

Перед восстановлением нужно сохранить или понять текущие незакоммиченные правки

Git помогает с текстовым файлом, но не заменяет backup всей системы

Branch нужен для безопасного эксперимента

main - стабильная версия

test-openvpn-route - экспериментальная ветка

git switch -с test-openvpn-route - создать и перейти в ветку

git switch main - вернуться в main

git merge test-openvpn-route - объединить ветку с main

Merge conflict объясняем без внутренней механики Git

Если две ветки изменили одно место, Git может потребовать ручного решения

Администратор открывает файл и оставляет правильный вариант

После исправления: `git add file`

Затем выполнить: `git commit`

Fast-Forward и Merge Commit internals не являются обязательной темой

Базовая ветка, эксперимент и merge



`git switch -c
experiment`



изменения
и проверка



`git switch main`



`git merge
experiment`



Идея:

Сначала проверяют изменения
в отдельной ветке,
затем объединяют их с **main**.

Репозиторий не держат прямо в /etc

Git Repository → отдельно

Validation → проверка синтаксиса

Test Environment → тест

Deployment → live /etc

Ветки и эксперименты не должны случайно менять рабочую конфигурацию сервера

5. .gitignore снижает риск случайного добавления

*~

*.swp

*.bak

*.dump

*.pcap и *.pcapng

.env, secrets/, *.key

.gitignore не является системой безопасности

.gitignore снижает риск случайного добавления

.gitignore не защищает уже добавленный файл

.gitignore не шифрует секреты

.gitignore не удаляет секрет из истории

Перед commit все равно нужны `git status` и `git diff --staged`

Уже отслеживаемый файл нужно убирать отдельно

```
git rm --cached secrets/client.key
```

Команда убирает файл из индекса Git

Файл остается в рабочем каталоге

После этого добавить `secrets/` и `*.key` в `.gitignore`

Если реальный секрет уже попал в commit, он остается в истории

6. Секреты нельзя хранить в Git

password

token

API key

private key

PSK

VPN profile with embedded key

SNMP community

Если secret попал в Git, его считают скомпрометированным

Secret попал в Git → считать скомпрометированным

Пароль нужно заменить

Token нужно перевыпустить

Private key нужно отозвать или заменить

VPN profile with embedded key нужно отозвать через
процедуру VPN

Удаление файла из последнего commit не всегда
решает проблему истории

Простой поиск секретов перед commit

```
grep -RInE 'password|passwd|token|secret|api[_-]  
]?key|private[_-]?key' .
```

```
grep -RIn 'BEGIN .*PRIVATE KEY' .
```

Это базовая самопроверка перед commit

Она не гарантирует идеальный secret scanning

Для junior достаточно начать с простых признаков и
внимательного git diff --staged

Файлы-примеры заменяют реальные секреты

.env.example можно хранить в Git

secrets.conf.example можно хранить в Git

VPN_USERNAME=CHANGE_ME

VPN_PASSWORD=CHANGE_ME

Реальный .env не хранить в Git

Пример показывает структуру, но не раскрывает реальные credentials

Git хранит не все метаданные системного файла

Git хранит содержимое файла

Git хранит путь файла

Git хранит executable bit

Git не является полноценной копией owner, permissions, SELinux и ACL

После deployment права и владельца нужно задавать отдельно

README должен объяснять применение конфигурации

Куда копировать файл

Какие права нужны

Как проверить синтаксис

Как применить изменение

Как выполнить rollback

Какая служба зависит от этого файла

7. Remote Repository хранит общую удаленную историю

Local Repository - репозиторий на текущем компьютере

Remote Repository - общая удаленная история

```
git remote add origin ssh://git@git.example.test/linux-  
infrastructure.git
```

```
git remote -v
```

```
git push -u origin main
```

Remote упрощает совместную работу и хранение истории вне одной машины

fetch и pull нужно различать

git fetch получает изменения без автоматического изменения текущей ветки

git pull получает и интегрирует изменения

Для junior важно понимать, что pull меняет рабочую ветку

rebase в этой лекции не разбирается подробно

Перед pull полезно выполнить git status

Дополнительно: tag отмечает принятую версию

Tag можно использовать для отметки принятой версии лаборатории

Например, `v1-server-health-lab`

Tags полезны для релизов и контрольных точек

В этой лекции tag не является обязательной практикой

Сначала студент должен уверенно выполнять базовый `commit workflow`

Break/Fix: секрет случайно попал в Staging Area

Студент случайно выполняет `git add` .

В Staging попадает `secrets/client.key`

Проверка: `git status`

Исправление: `git restore --staged secrets/client.key`

Добавить в `.gitignore`: `*.key` и `secrets/`

Повторная проверка: `git status`

Итоговый практический сценарий: создать и зафиксировать

Создать linux-infrastructure и выполнить git init

Создать структуру каталогов

Добавить scripts/server-health.sh

Выполнить bash -n scripts/server-health.sh

Посмотреть git diff

Добавить файл и проверить git diff --staged

Итоговый практический сценарий: проверить и отправить

Создать commit с понятным сообщением

Посмотреть `git log --oneline` и `git show HEAD`

Изменить файл и выполнить `git restore`

Создать `.gitignore`

Проверить случайный секрет

Отправить repository в Remote



Безопасный Git workflow администратора

Изменение → проверка → фиксация → история

1



Изменить
файл

scripts/server-health.sh
configs/nginx/app.conf

2



git status

что изменилось

3



git diff

проверить
изменения

4



Проверить

```
bash -n /  
nginx -t /  
sshd -t
```

5



git add

добавить только
нужные файлы

6



git diff
--staged

что войдёт
в commit

7



git commit
понятное сообщение

Add server
health check

8



git log /
git show

проверить
историю



git push
отправить
в remote



Правила



Сначала status и diff



Сначала проверка,
потом commit



Не использовать
бездумно git add .



Не хранить пароли,
токены и private keys



Главный принцип: сначала проверить изменение, затем зафиксировать его в Git и при необходимости быстро восстановить предыдущую версию.



Итоги лекции

Git помогает администратору фиксировать историю текстовых изменений

Working Tree, Staging Area и Repository нельзя смешивать

Перед commit нужно проверить файл и прочитать diff

.gitignore снижает риск, но не защищает секреты сам по себе

Git не является backup и не заменяет config management

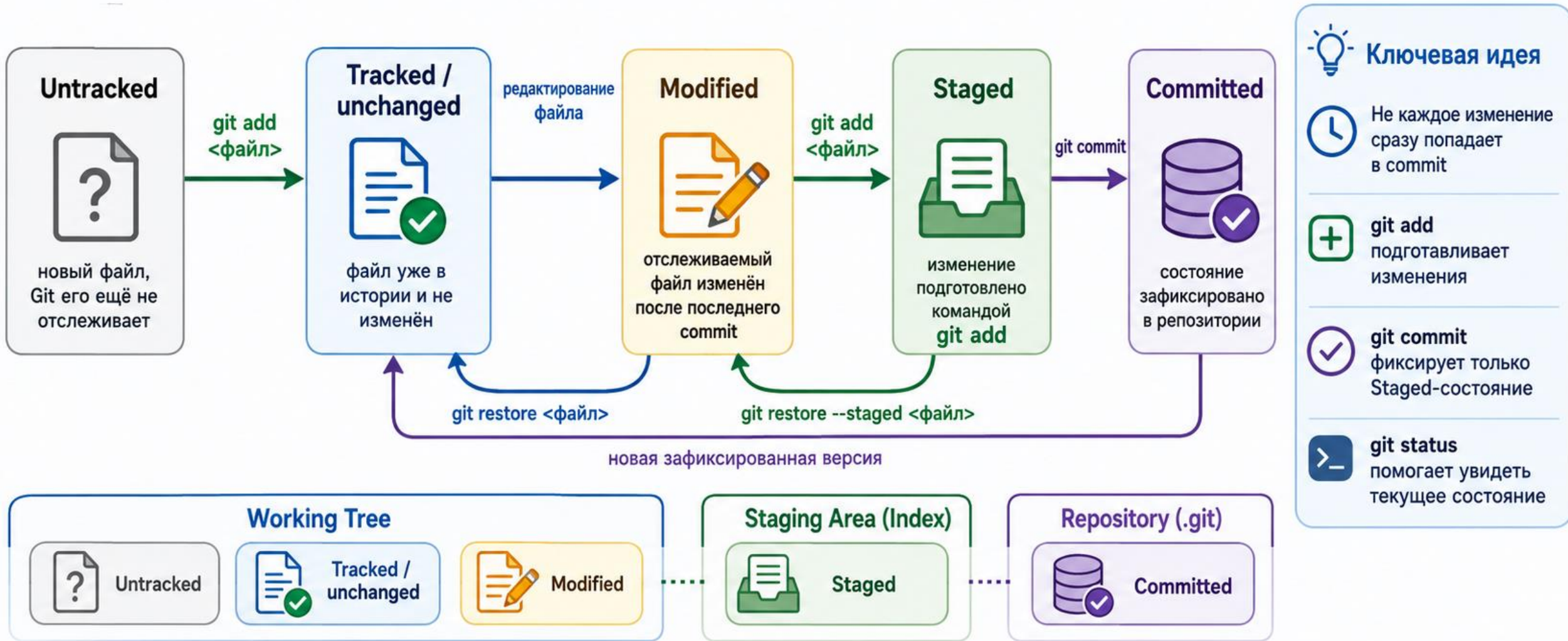
Главный навык junior - безопасно создать commit и не занести лишнее

Контрольные вопросы

1. Для чего администратору Git?
2. Чем Working Tree отличается от Staging Area?
3. Что показывает git status?
4. Что показывает git diff?
5. Что делает git add?
6. Что такое commit?
7. Как отменить незакоммиченные изменения?
8. Для чего нужен .gitignore?
9. Почему нельзя хранить private keys и passwords?
10. Почему Git не является Backup?

Схема: состояния файла в Git

Как файл перемещается между состояниями в Working Tree, Staging Area и Repository



Важно

Частая ошибка новичка: думать, что `git commit` сохраняет все изменения автоматически. На самом деле `commit` берёт только то, что находится в Staging Area.

Диаграмма: ветка, merge и конфликт

Ветки позволяют работать независимо. Merge объединяет изменения.

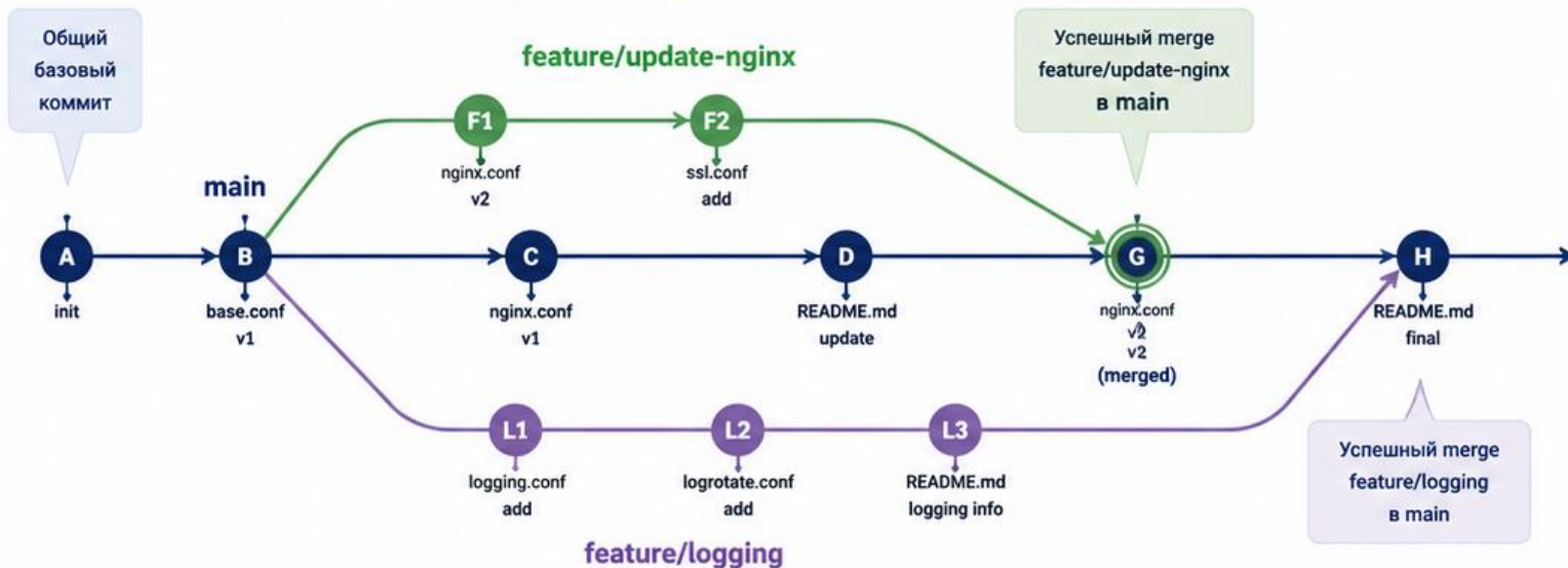
Если изменения затрагивают одни и те же строки — возникает конфликт.

Условные обозначения

- main (production)
- feature/update-nginx
- feature/logging

- ✓ Успешный merge
- ⚠ Конфликт

Как развиваются ветки



Конфликт при merge

Обе ветки изменили одни и те же строки в nginx.conf

main (D)

```
1 server {
2   listen 80;
3   server_name site.local;
4 }
```

feature/update-nginx (F1)

```
1 server {
2   listen 8080; # изменено
3   server_name site.local;
4 }
```

Конфликт

```
$ git merge feature/update-nginx
Auto-merging nginx.conf
CONFLICT (content): Merge conflict in nginx.conf
Automatic merge failed; fix conflicts and then commit the result.
```

Разрешение конфликта

- 1 Открой файл
 - 2 Добавь файл
 - 3 Закоммить merge
- Исправь конфликт вручную
- git add nginx.conf
- git commit (сообщение о merge)

• Итоговый файл после разрешения

```
1 server {
2   listen 8080;
3   server_name site.local;
4 }
```

✓ Конфликт разрешён.
История сохранена.

Что даёт работа с ветками

- Изоляция изменений**
Работайте над задачей, не влияя на main
- Безопасные эксперименты**
Можно пробовать идеи и легко откатиться
- Понятная история**
Каждый шаг описан в коммитах
- Совместная работа**
Легко делиться, проверять и обсуждать
- Надёжность**
Merge и история предотвращают потери

Полезные команды

```
$ git branch # список веток
$ git branch <name> # создать ветку
$ git checkout <branch> # переключиться на ветку
$ git checkout -b <name> # создать и переключиться
$ git merge <branch> # объединить ветку в текущую
$ git log --graph --oneline --all # граф истории
```

Стандартный поток работы



Рекомендации

- ✓ Делайте маленькие и понятные commit
- ✓ Пишите осмысленные сообщения commit
- ✓ Проверяйте код перед merge (тесты, линтеры)
- ✓ Не храните секреты и ключи в репозитории
- ✓ Регулярно синхронизируйте ветки с main

Главное

Ветки помогают работать независимо. Merge объединяет изменения. Конфликты — нормальная часть совместной работы.